```
# Structure-only: vertices = the 240 elements; edge = relation value -1; simplices =
complete subgraphs.
# Input: the 240 elements (standard coordinates; REVIEW-origin, not derived from T).
Exact integer arithmetic via doubled coordinates.
import numpy as np, itertools
rt=[]
for i,j in itertools.combinations(range(8),2):
    for si in (2,-2):
        for sj in (2,-2):
            v=np.zeros(8,int); v[i]=si; v[j]=sj; rt.append(v)
for s in itertools.product([1,-1],repeat=8):
    if sum(x<0 for x in s)%2==0: rt.append(np.array(s))
R=np.array(rt); G=(R@R.T)//4          # relation values in {-2..2}
print("elements:",len(R)," relation table symmetric (carries no
direction):",bool((G==G.T).all()))
A=(G==-1).astype(int); n=len(R)
E=int(A.sum()//2); print("edges (relation value -1):",E," degree of each
element:",sorted(set(A.sum(1))))
# triangles = triples pairwise -1 ; 4-cliques
tri=[(i,j,k) for i in range(n) for j in np.where(A[i])[0] if j>i for k in
np.where(A[i]&A[j])[0] if k>j]
print("triangles:",len(tri))
print("every triangle sums to zero:",all(not (R[i]+R[j]+R[k]).any() for i,j,k in
tri))
print("each edge lies in exactly",set(int((A[i]&A[j]).sum()) for i in range(n) for j
in np.where(A[i])[0] if j>i),"triangle(s)")
c4=sum(1 for i,j,k in tri for l in np.where(A[i]&A[j]&A[k])[0] if l>k); print("4-
element complete subgraphs:",c4)
print("Euler characteristic V-E+F =",n-E+len(tri))
print("directions: each edge has 3 states (either way / both ways) -> 3^E
assignments; each triangle has 2 cyclic orientations -> 2^%d orientation
choices"%len(tri))
# height orientation (generic direction): source/sink
g=np.random.RandomState(0).randn(8); h=R@g; print("generic height ordering: all
heights distinct:",len(set(np.round(h,9)))==n," source = element of lowest height,
sink = highest;  sink == -
source:",bool(((R[np.argmax(h)]+R[np.argmin(h)])==0).all()))
bad=0
for i,j,k in tri:
    t=(i,j,k); out=sorted(sum(1 for b in t if b!=a and h[a]<h[b]) for a in t)   #
out-degree inside the triangle when each edge points low->high
    if out!=[0,1,2]: bad+=1
```

```
print("triangles whose height orientation is NOT a transitive triple
(computed):",bad,"of",len(tri))
```